浙江大学实验报告

姓名：金宣妤　　专业：电子科学与技术　　学号：3240100333

课程名称：Signals and Systems　　任课老师：Prof. Xiaoming Chen

实验名称：Lecture2: Linear Time-Invariant Systems　　实验日期：2026.3.31

1 Experimental Objectives

(1) Learn how to use basic MATLAB functions (like filter, lsim, step, impulse) to simulate continuous-time and discrete-time systems.

(2) Use MATLAB to test if a discrete-time system is linear (satisfies the superposition principle) and time-invariant.

(3) Calculate the mathematical impulse and step responses of continuous-time LTI systems. Understand how to use a short pulse to approximate an ideal Dirac impulse.

(4) Understand the math behind inverse filtering and use it to remove echoes from real speech signals.

2 Experimental Principles

(1) Basic Properties of LTI Systems

An LTI system is both linear and time-invariant. "Linear" means it follows the superposition principle (the output of combined inputs equals the combined outputs). "Time-invariant" means if we delay the input, the output is delayed by the exact same amount, but its shape does not change.

(2) System Description

We use differential equations for continuous LTI systems and difference equations for discrete LTI systems. An LTI system is fully defined by its unit impulse response, h(t) or h[n].

(3) Ideal Impulse and Pulses

The ideal Dirac impulse $\delta(t)$ is infinitely high, infinitely narrow, and has an area of 1. In real life, we can use a very short pulse $\delta^{\Delta}(t)$ to simulate it, but this pulse must keep an area of 1 .

(4) Echo and Inverse System

An echo is just the original signal plus a delayed and weaker version of itself: $y[n] = x[n] + \alpha x[n-N]$. To remove the echo, we build an "inverse system" using a feedback loop: $z[n] + \alpha z[n-N] = y[n]$. This recovers the original signal x[n] .

3 Experimental Contents, Results, and Analysis

3.1 Application of “filter”

When using the **filter(b, a, x)** function in MATLAB to solve difference equations, we must put all y terms on the left side of the equation and all x terms on the right side. Vector a holds the coefficients of y, and vector b holds the coefficients of x, ordered by time delay 0, 1, 2....

Problem:

To specify the system described by the difference equation $y[n] + 2y[n-1] = x[n] + 3x[n-1]$, define these vectors as **a**=[1 2] and **b**=[1 3].

- Define coefficient vectors **a1** and **b1** to describe the causal LTI system specified by $y[n] = 0.5x[n] + x[n-1] + 2x[n-2]$.
- Define coefficient vectors **a2** and **b2** to describe the causal LTI system specified by $y[n] = 0.8y[n-1] + 2x[n]$.
- Define coefficient vectors **a3** and **b3** to describe the causal LTI system specified by $y[n] - 0.8y[n-1] = 2x[n-1]$.
- For each of these three systems, use **filter** to compute the response $y[n]$ on the interval $1 \leq n \leq 4$ to the input signal $x[n] = nu[n]$. You should begin by defining the vector x=[1 2 3 4], which contains x[n] on the $1 \leq n \leq 4$ interval.

Code:

```
clear;clc;
n_filter = 1:4;
x = [1, 2, 3, 4];

a1 = 1;
b1 = [0.5, 1, 2];
y1 = filter(b1, a1, x);

a2 = [1, -0.8];
b2 = 2;
y2 = filter(b2, a2, x);

a3 = [1, -0.8];
b3 = [0, 2];
y3 = filter(b3, a3, x);

disp(['y1 = ', num2str(y1)]);
disp(['y2 = ', num2str(y2)]);
disp(['y3 = ', num2str(y3)]);
```

Output:

```
命令行窗口
y1 = 0.5          2         5.5           9
y2 = 2          5.6       10.48       16.384
y3 = 0            2         5.6        10.48
```

The output numbers from the filter function perfectly match the math calculations of the difference equations.

y3 is y2 delayed by one position. This proves that the system is Time-Invariant. A delay in the input only causes the exact same delay in the output, without changing the values.

3.2 Application of "lsim"

(1)Problem:

- Use **lsim** to compute the response of the causal LTI system described by

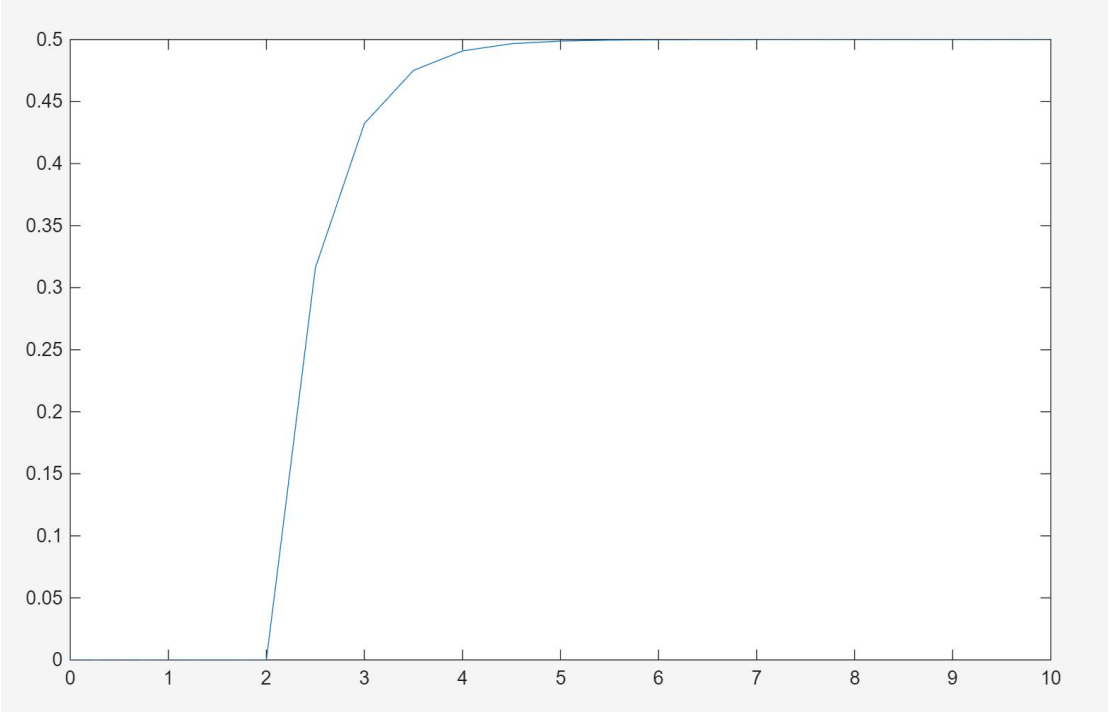
$$\frac{dy(t)}{dt} = -2y(t) + x(t), \tag{7}$$

to the input $x(t) = u(t-2)$, using t=[0:0.5:10].

code:

```
clear;clc;

a = [1, 2];
b = 1;

t = 0:0.5:10;
x = (t >= 2) * 1;
y_sim = lsim(b, a, x, t);

plot(t, y_sim);
```

Output:



For the system

$$\frac{dy(t)}{dt} + 2y(t) = x(t)$$

a delayed unit step input $u(t-2)$ was generated using the logical condition t1 >= 2. The lsim simulation shows that the system output starts exactly at $t = 2$. This fits the behavior of a causal system, proving that the output never appears before the input.

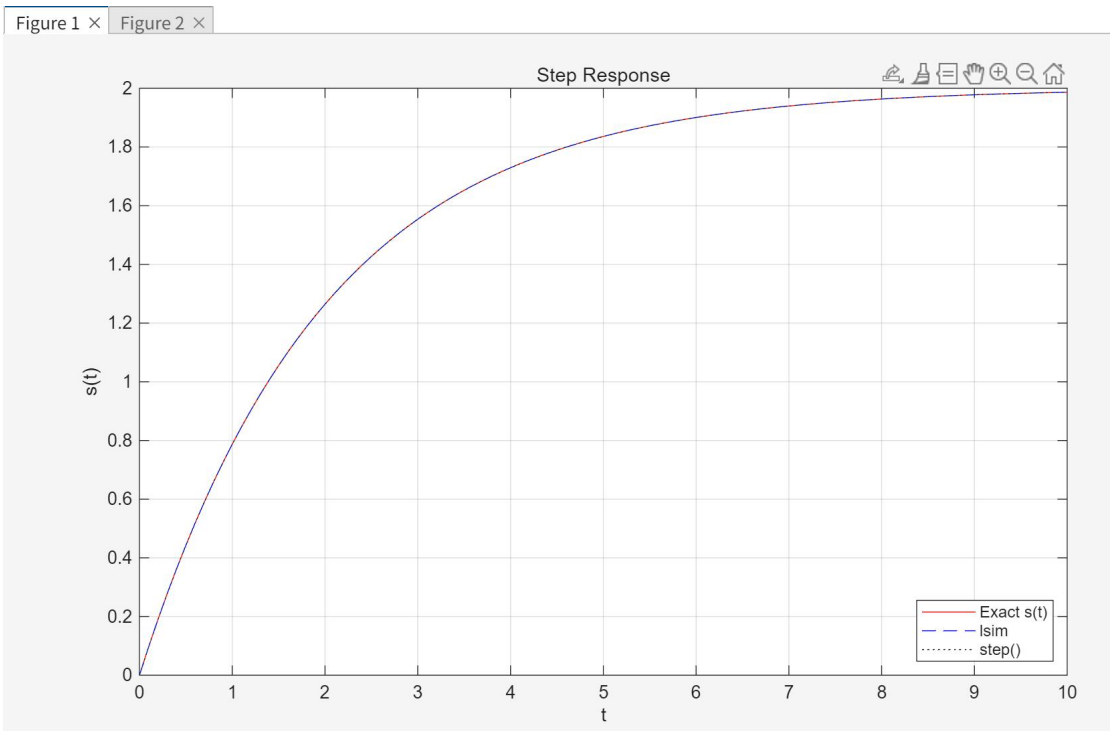
(2)Problem:

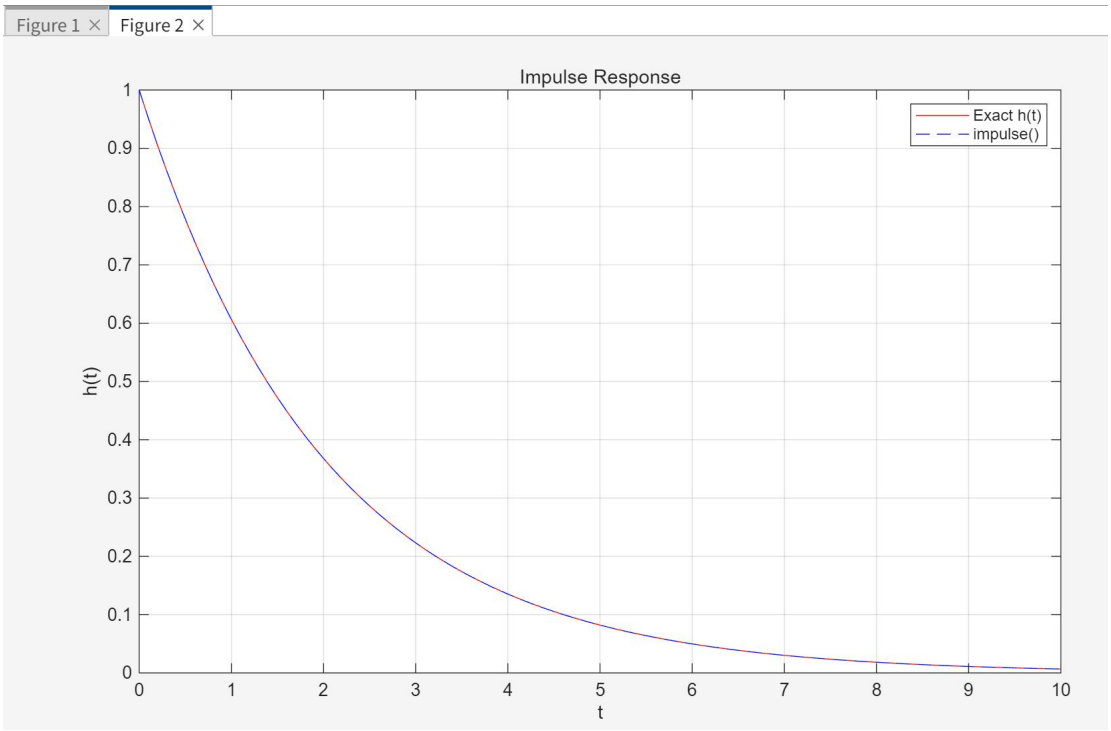
- The functions **impulse** and **step** can be used to compute the impulse and step response of such systems. Use them to compute the step and impulse responses of the causal LTI system characterized by Eq. (5). Compare the step response computed by **step** with that computed by **lsim**. Compare the signal returned by **impulse** with the exact impulse response given by the derivative of $s(t)$ in Eq. (6).

code:

```
clear; clc; close all;

a5 = [1, 0.5];
b5 = 1;
t5 = 0:0.1:10;

s_exact = 2 * (1 - exp(-t5/2));
y_lsim = lsim(b5, a5, ones(size(t5)), t5);
y_step = step(b5, a5, t5);

figure(1);
plot(t5, s_exact, 'r-', t5, y_lsim, 'b--', t5, y_step, 'k:');
title('Step Response');
legend('Exact s(t)', 'lsim', 'step()', 'Location', 'Southeast');
xlabel('t'); ylabel('s(t)'); grid on;

h_exact = exp(-t5/2);
y_impulse = impulse(b5, a5, t5);
figure(2);
plot(t5, h_exact, 'r-', t5, y_impulse, 'b--');
title('Impulse Response');
legend('Exact h(t)', 'impulse()');
xlabel('t'); ylabel('h(t)'); grid on;
```

Output:





Step Response Verification: For the system

$$\frac{dy(t)}{dt} + 0.5y(t) = x(t)$$

the exact theoretical solution $s(t) = 2(1 - e^{-t/2})u(t)$ is plotted alongside the numerical results from MATLAB's lsim and step functions. The three curves perfectly overlap.

Impulse Response Verification: According to LTI system theory, the unit impulse response h(t) is the derivative of the step response s(t). Comparing the analytical derivative $h(t) = e^{-t/2}u(t)$ with the output of the impulse function shows a perfect match. This not only verifies the simulation function but also visually proves the calculus relationship between impulse and step responses.

3.3 problem1

Consider the following three systems:

System 1: $w[n] = x[n] - x[n-1] - x[n-2]$,
System 2: $y[n] = \cos(x[n])$,
System 3: $z[n] = n\,x[n]$,

where $x[n]$ is the input to each system, and $w[n]$, $y[n]$, and $z[n]$ are the corresponding outputs.

(a). Consider the three inputs signals $x_1[n] = \delta[n]$, $x_2[n] = \delta[n-1]$, and $x_3[n] = (\delta[n] + 2\,\delta[n-1])$. For System 1, store in `w1`, `w2`, and `w3` the responses to the three inputs. The vectors `w1`, `w2`, and `w3` need to contain the values of $w[n]$ only on the interval $0 \le n \le 5$. Use `subplot` and `stem` to plot the four functions represented by `w1`, `w2`, `w3`, and `w1+2*w2` within a single figure. Make analogous plots for Systems 2 and 3.

(b). State whether or not each system is linear. If it is linear, justify your answer. If it is not linear, use the signals plotted in Part (a) to supply a counter-example.

(c). State whether or not each system is time-invariant. If it is time-invariant, justify your answer. If it is not time-invariant, use the signals plotted in Part (a) to supply a counter-example.

Code:

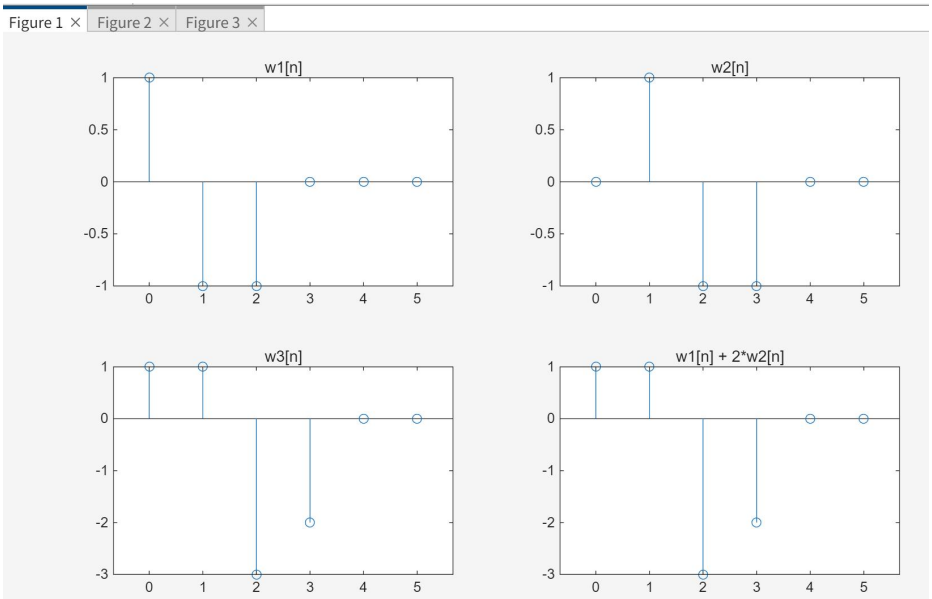
```
clear; clc; close all;
n = 0:5;
x1 = [1, 0, 0, 0, 0, 0];
x2 = [0, 1, 0, 0, 0, 0];
x3 = x1 + 2 * x2;

% 1
b = [1, -1, -1]; a = 1;
w1 = filter(b, a, x1);
w2 = filter(b, a, x2);
w3 = filter(b, a, x3);

figure(1);
subplot(2,2,1); stem(n, w1); title('w1[n]');
subplot(2,2,2); stem(n, w2); title('w2[n]');
subplot(2,2,3); stem(n, w3); title('w3[n]');
subplot(2,2,4); stem(n, w1 + 2.*w2); title('w1[n] + 2*w2[n]');

% 2
y1 = cos(x1); y2 = cos(x2); y3 = cos(x3);

figure(2);
subplot(2,2,1); stem(n, y1); title('y1[n]');
subplot(2,2,2); stem(n, y2); title('y2[n]');
subplot(2,2,3); stem(n, y3); title('y3[n]');
subplot(2,2,4); stem(n, y1 + 2.*y2); title('y1[n] + 2*y2[n]');

% 3
z1 = n .* x1;   z2 = n .* x2;   z3 = n .* x3; %点乘

figure(3);
subplot(2,2,1); stem(n, z1); title('z1[n]');
subplot(2,2,2); stem(n, z2); title('z2[n]');
subplot(2,2,3); stem(n, z3); title('z3[n]');
subplot(2,2,4); stem(n, z1 + 2.*z2); title('z1[n] + 2*z2[n]');
```

This code uses plots to visually test the linearity and time-invariance of three systems.

Linearity: Compare subplot 3 (response to combined input x_3) and subplot 4 (combined responses $y_1 + 2y_2$). If they are exactly the same, it follows the superposition principle and is a linear system.

Time-Invariance: Compare subplot 1 (response to x_1) and subplot 2 (response to delayed input x_2). If subplot 2 is just subplot 1 shifted to the right by one step, it is a time-invariant system.

(1) System 1:

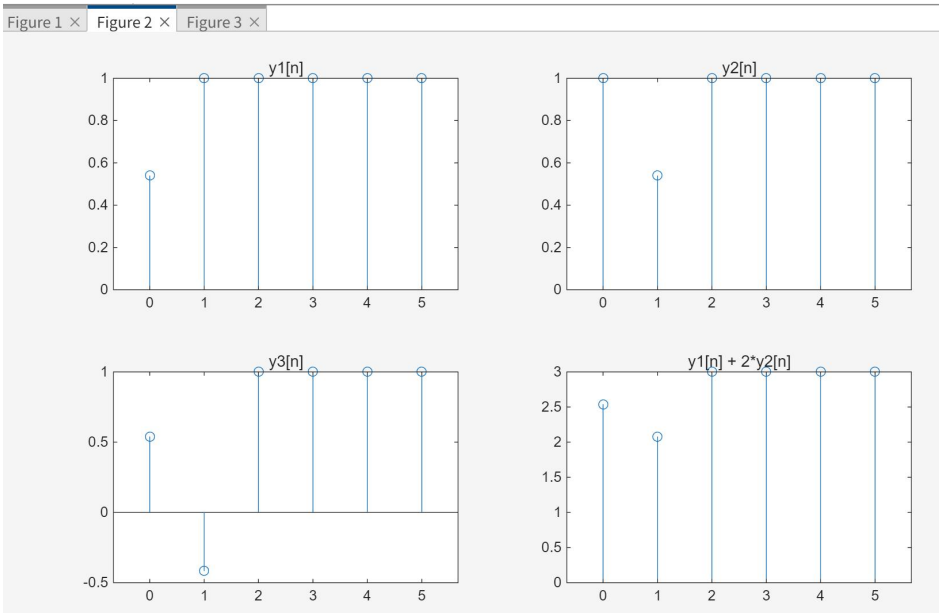


Linearity: Subplot 3 and 4 match . It is linear.

Time-Invariance: Subplot 2 is exactly subplot 1 shifted right. It is time-invariant.

Conclusion: It is a Linear Time-Invariant (LTI) system.

(2) System 2:

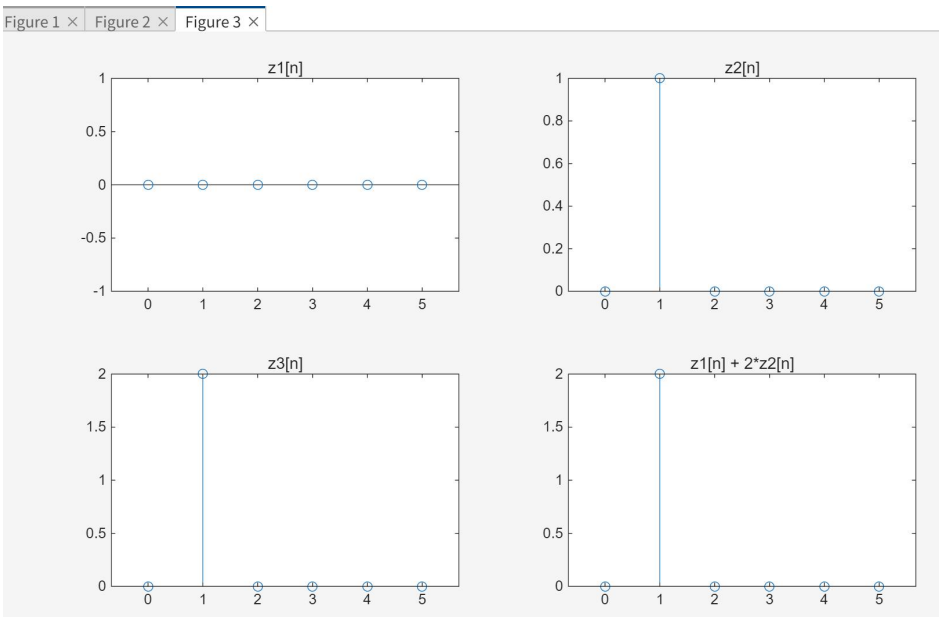


Linearity: Subplot 3 and 4 are different. It is non-linear.

Time-Invariance: Subplot 2 is still subplot 1 shifted right. It is time-invariant.

Conclusion: It is a non-linear but time-invariant system.

(3) Sy$stem3$:



Linearity: Subplot 3 and 4 match perfectly. It is linear.

Time-Invariance: Subplot 2 is not a simple shift of subplot 1. It is time-varying.

Conclusion: It is a linear but time-varying system.

3.4 problem2

Problem 2

Consider the causal LTI system whose input $x(t)$ and output $y(t)$ satisfy the following first-order linear differential equation

$$\frac{dy(t)}{dt} + 3y(t) = x(t). \quad \text{Eq.(2.20)}$$

(a). Analytically derive the unit step response $s(t)$ and the unit impulse response $h(t)$ of the causal LTI system defined by Eq. (2.20). Hint: The impulse response is equal to $ds(t)/dt$. Store in `h` and `s` the values of $h(t)$ and $s(t)$ at each time sample in the vector `t=[-1:0.05:4]`. Plot both `h` and `s` versus `t`.

(b). Use `step` and `impulse` to verify the functions that you computed in Part (a). The functions `step(b,a,t)` and `impulse(b,a,t)` are explained in Tutorial 2.3, the tutorial for `lsim`. The vectors `b` and `a` are determined by the coefficients in Eq. (2.20). For the time vector `t`, use `t=[0:0.05:4]`.

(c). Use `lsim` to simulate the response to $\delta^{\Delta}(t)$ for $\Delta = 0.1$, 0.2, and 0.4. For the time vector, use `t=[-1:0.05:4]`, and store in `d1`, `d2`, and `d3` the corresponding values of $\delta^{\Delta}(t)$ for $\Delta = 0.1$, 0.2, and 0.4, respectively. Remember that $\delta^{\Delta}(\Delta) = 0$, not $1/\Delta$. Store in `h1`, `h2`, and `h3` the simulated response for $\Delta = 0.1$, 0.2, and 0.4, respectively. These vectors contain the simulated values of $h^{\Delta}(t)$ at the time samples given in `t`.

On three separate figures, plot the simulated values of $h^{\Delta}(t)$. On each figure, also include a plot of $h(t)$, which was stored in h in Part (a). How does $h^{\Delta}(t)$ compare to $h(t)$ as Δ decreases?

(d). Why is it necessary to ensure that the pulse $\delta^{\Delta}(t)$ has unit area? Namely, what if the response to

$$D^{\Delta}(t) = \begin{cases} 1, & 0 \le t < \Delta, \\ 0, & \text{otherwise}, \end{cases}$$

13

analytical derivation

$\frac{dy(t)}{dt} + 3y(t) = x(t)$ ，令 $x(t) = u(t)$

拉氏变换 $\Rightarrow$ $sY(s) + 3Y(s) = \frac{1}{s}$

$\Rightarrow Y(s) = \frac{1}{s(s+3)} = \frac{1}{3}\left(\frac{1}{s} - \frac{1}{s+3}\right)$

$\Rightarrow y(t) = \frac{1}{3}(1 - e^{-3t})$

即 $s(t) = \frac{1}{3}(1 - e^{-3t})u(t)$

$h(t) = \frac{ds(t)}{dt} = e^{-3t} * u(t)$

Code in (a)(b)

```
clear; clc; close all;

t = -1:0.05:4;
u = (t >= 0);

s = (1/3) * (1 - exp(-3.*t)) .* u;
h = exp(-3.*t) .* u;

figure(1);
subplot(2,1,1); plot(t, s); title('s(t)');
subplot(2,1,2); plot(t, h); title('h(t)');

a = [1, 3];
b = 1;
t2 = 0:0.05:4;

figure(2);
subplot(2,1,1); step(b, a, t2); title('step()');
subplot(2,1,2); impulse(b, a, t2); title('impulse()');
```

Figure for (a)

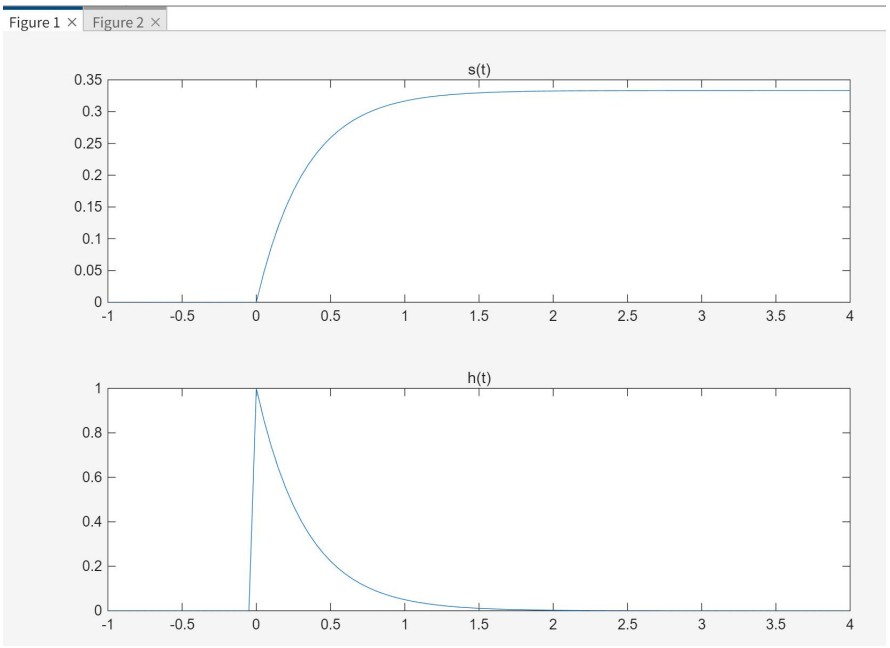
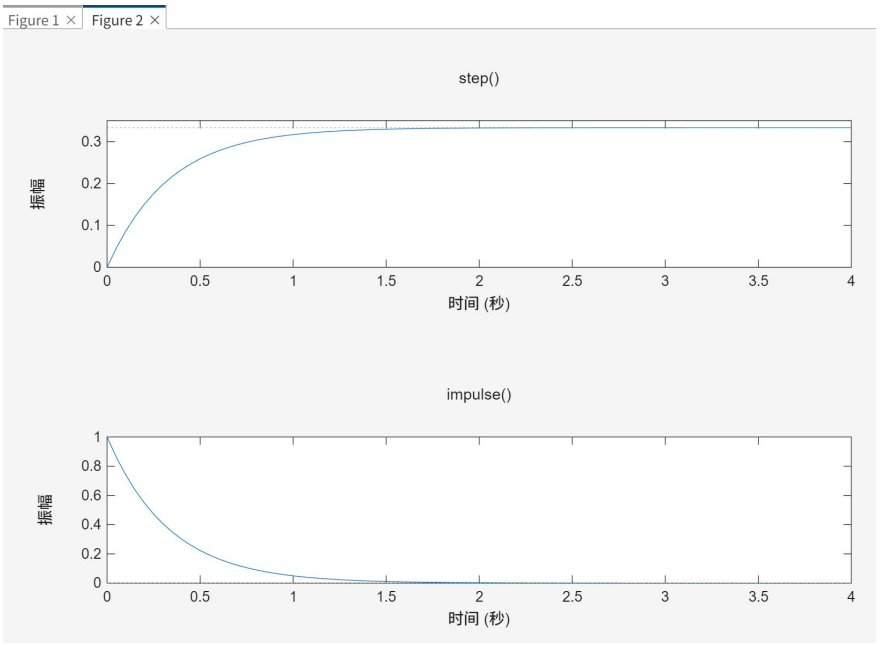


Figure for (b)



The perfect match proves that our math calculation for the differential equation is correct.

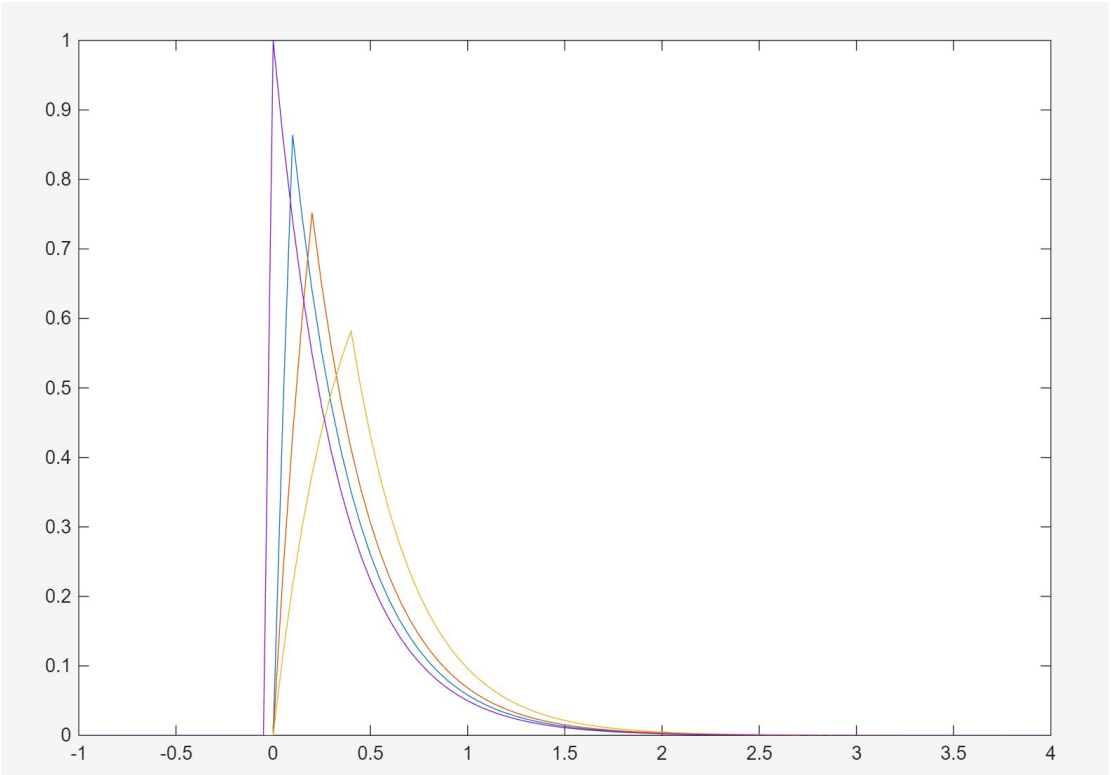
It proves that for continuous-time LTI systems, the impulse response is indeed the derivative of the step response($h(t) = \frac{ds(t)}{dt}$). The step response rises fastest at $t = 0$, which exactly matches the highest point (amplitude 1) of the impulse response.

It shows that MATLAB's step and impulse functions are highly accurate and reliable for analyzing LTI systems.

Code for (c)

```
clear; clc; close all;

delta = 0.05;
t = -1:0.05:4;
a = [1 3];
b = 1;

dt1 = 0.1;
dt2 = 0.2;
dt3 = 0.4;

d1 = zeros(1, length(t));
d1(1, 1/delta+1 : 1/delta+dt1/delta) = 1/dt1;

d2 = zeros(1, length(t));
d2(1, 1/delta+1 : 1/delta+dt2/delta) = 1/dt2;

d3 = zeros(1, length(t));
d3(1, 1/delta+1 : 1/delta+dt3/delta) = 1/dt3;

u = (t >= 0);
h = exp(-3.*t) .* u;

h1 = lsim(b, a, d1, t);
h2 = lsim(b, a, d2, t);
h3 = lsim(b, a, d3, t);

plot(t, h1); hold on;
plot(t, h2); hold on;
plot(t, h3); hold on;
plot(t, h);
```

output for (c)



Answer for (c):

As the pulse width Δ decreases $(0.4 \rightarrow 0.2 \rightarrow 0.1)$, the input pulse looks more like an ideal Dirac impulse $\delta(t)$. Therefore, the simulated output $h^{\Delta}(t)$ gets much closer and finally converges to the true unit impulse response h(t).

Answer for (d)

Why Keep Unit Area: When creating the pulses, we set the height to $1/\Delta$ to make sure the area is always 1. This is necessary because the true unit impulse response h(t) is based on an ideal impulse $\delta(t)$ which has an integral (area) of 1. Because this is a linear system, it follows the rule of scaling. If the input pulse area is not 1 (for example, if the area is only Δ), the output amplitude will also shrink by a factor of Δ. That small wave would not correctly show the true impulse behavior of the system .

3.5 problem3

Problem 3

In this exercise, you will consider the problem of removing an echo from a recording of a speech signal. This project will use the audio capabilities of MATLAB to play recordings of both the original speech and the result of your processing. To begin this exercise you will need to load the speech file `lineup.mat`, which is contained in the Computer Explorations Toolbox. The Computer Explorations Toolbox can be obtained from The MathWorks at the address provided in the Preface. If this speech file is already somewhere in your MATLABPATH, then you can load the data into MATLAB by typing

```
>> load lineup.mat
```

You can check your MATLABPATH, which is a list of all the directories which are currently accessible by MATLAB, by typing `path`. The file `lineup.mat` must be in one of these directories.

Once you have loaded the data into MATLAB, the speech waveform will be stored in the variable `y`. Since the speech was recorded with a sampling rate of 8192 Hz, you can hear the speech by typing

```
>> sound(y,8192)
```

You should hear the phrase "line up" with an echo. The signal $y[n]$, represented by the vector `y`, is of the form

$$y[n] = x[n] + \alpha x[n - N], \tag{2.21}$$

where $x[n]$ is the uncorrupted speech signal, which has been delayed by N samples and added back in with its amplitude decreased by $\alpha < 1$. This is a reasonable model for an echo resulting from the signal reflecting off of an absorbing surface like a wall. If a microphone is placed in the center of a room, and a person is speaking at one end of the room, the recording will contain the speech which travels directly to the microphone, as well as an echo which traveled across the room, reflected off of the far wall, and then into the microphone. Since the echo must travel further, it will be delayed in time. Also, since the speech is partially absorbed by the wall, it will be decreased in amplitude. For simplicity ignore any further reflections or other sources of echo.

For the problems in this exercise, you will use the value of the echo time, $N = 1000$, and the echo amplitude, $\alpha = 0.5$.

Basic Problems

(a). In this exercise you will remove the echo by linear filtering. Since the echo can be represented by a linear system of the form Eq. (2.21), determine and plot the impulse

response of the echo system Eq. (2.21). Store this impulse response in the vector **he** for $0 \leq n \leq 1000$.

(b). Consider an echo removal system described by the difference equation

$$z[n] + \alpha z[n - N] = y[n], \tag{2.22}$$

where $y[n]$ is the input and $z[n]$ is the output which has the echo removed. Show that Eq. (2.22) is indeed an inverse of Eq. (2.21) by deriving the overall difference equation relating $z[n]$ to $x[n]$. Is $z[n] = x[n]$ a valid solution to the overall difference equation?

Intermediate Problems

(c). The echo removal system Eq. (2.22) will have an infinite-length impulse response. Assuming that $N = 1000$, and $\alpha = 0.5$, compute the impulse response using **filter** with an input that is an impulse given by **d=[1 zeros(1,4000)]**. Store this 4001 sample approximation to the impulse response in the vector **her**.

(d). Implement the echo removal system using **z=filter(1,a,y)**, where **a** is the appropriate coefficient vector derived from Eq. (2.22). Plot the output using **plot**. Also, listen to the output using **sound**. You should no longer hear the echo.

Theoretical Proof (For Problem b):

Original System:$y[n] = x[n] + \alpha x[n - N]$.

Inverse System:$z[n] + \alpha z[n - N] = y[n]$.

Substitute y[n] into the inverse system equation. We get:

$$z[n] + \alpha z[n - N] = x[n] + \alpha x[n - N]$$

Verify the Solution:Assume the system works perfectly and $z[n] = x[n]$. This proves that the filter perfectly cancels the echo.

Code for (a)(c)(d)

```
clear; clc; close all;

alpha = 0.5;
N = 1000;

he = [1, zeros(1, N-1), alpha];

figure(1);
stem(0:N, he);

a = [1, zeros(1, N-1), alpha];
b = 1;

d = [1, zeros(1, 4000)];
n = 0:4000;
her = filter(b, a, d);

figure(2);
plot(n, her);

load lineup.mat

z = filter(b, a, y);

figure(3);
subplot(2,1,1); plot(y); title('Original');
subplot(2,1,2); plot(z); title('Processed');

sound(z, 8192);
```

Output for (a)

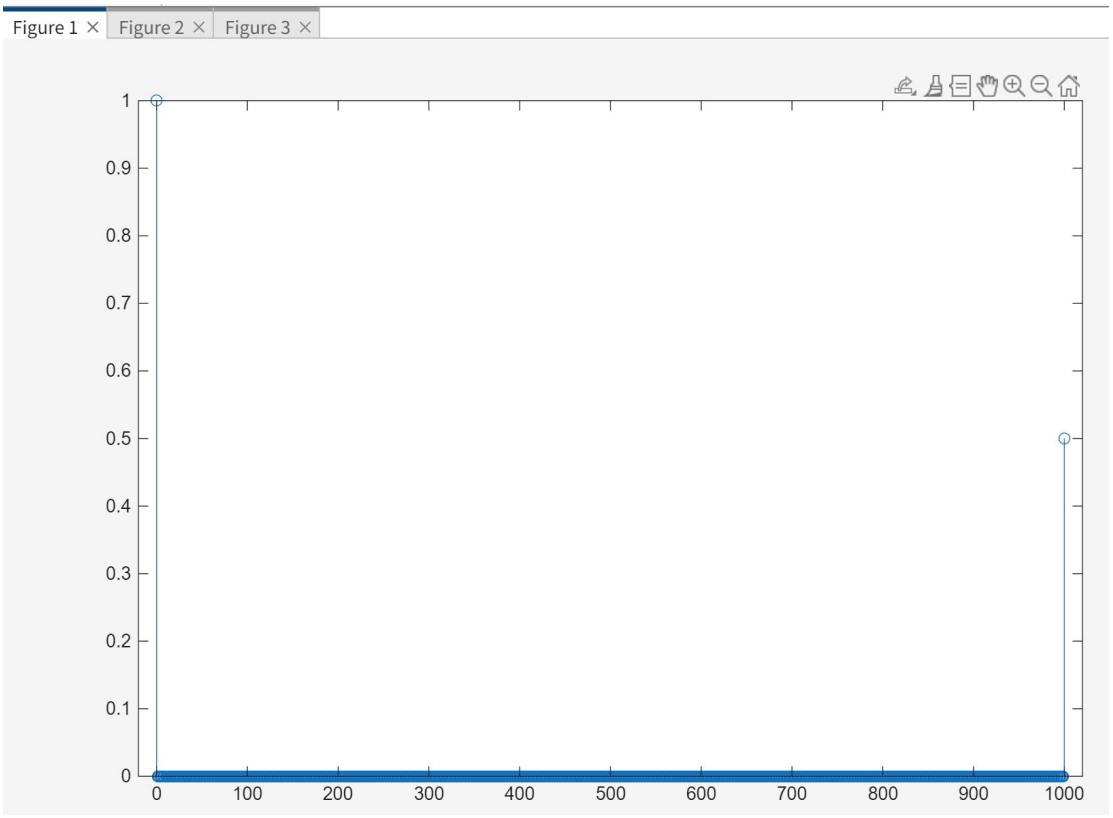


Figure 1 shows the impulse response he. It only has two spikes: the original sound at $n = 0$ (amplitude 1) and the echo at $n = 1000$ (amplitude 0.5).

Output for (c)

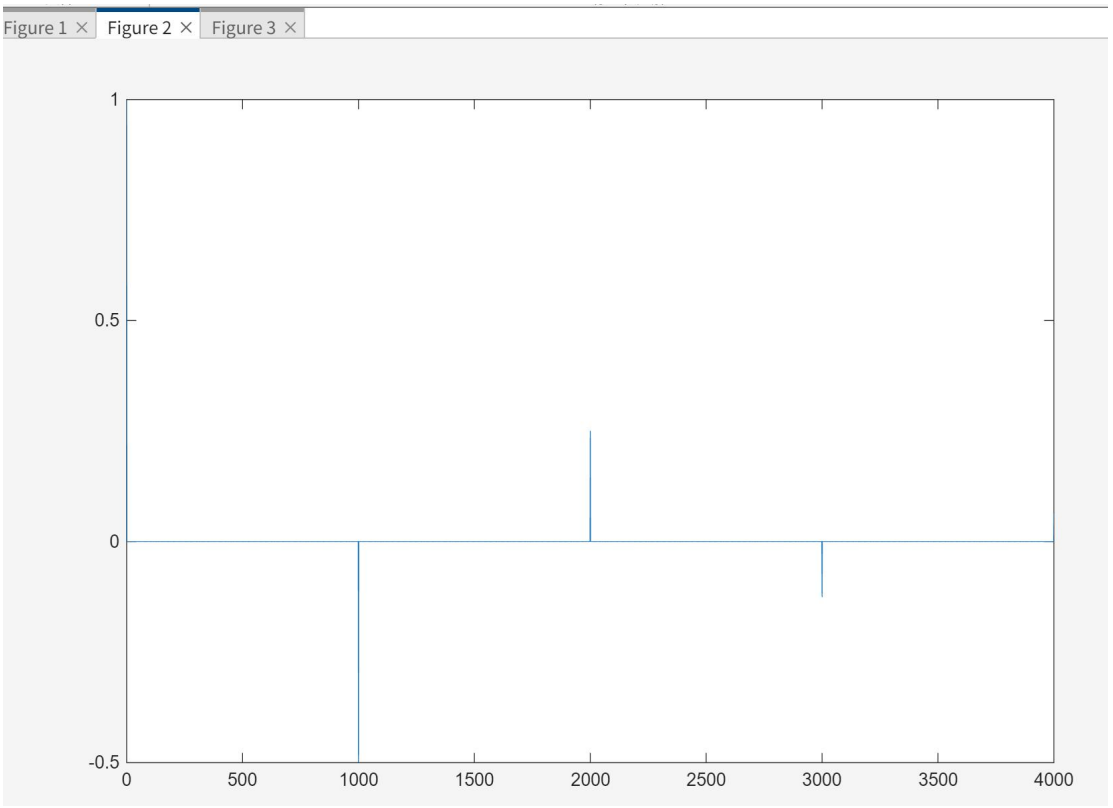
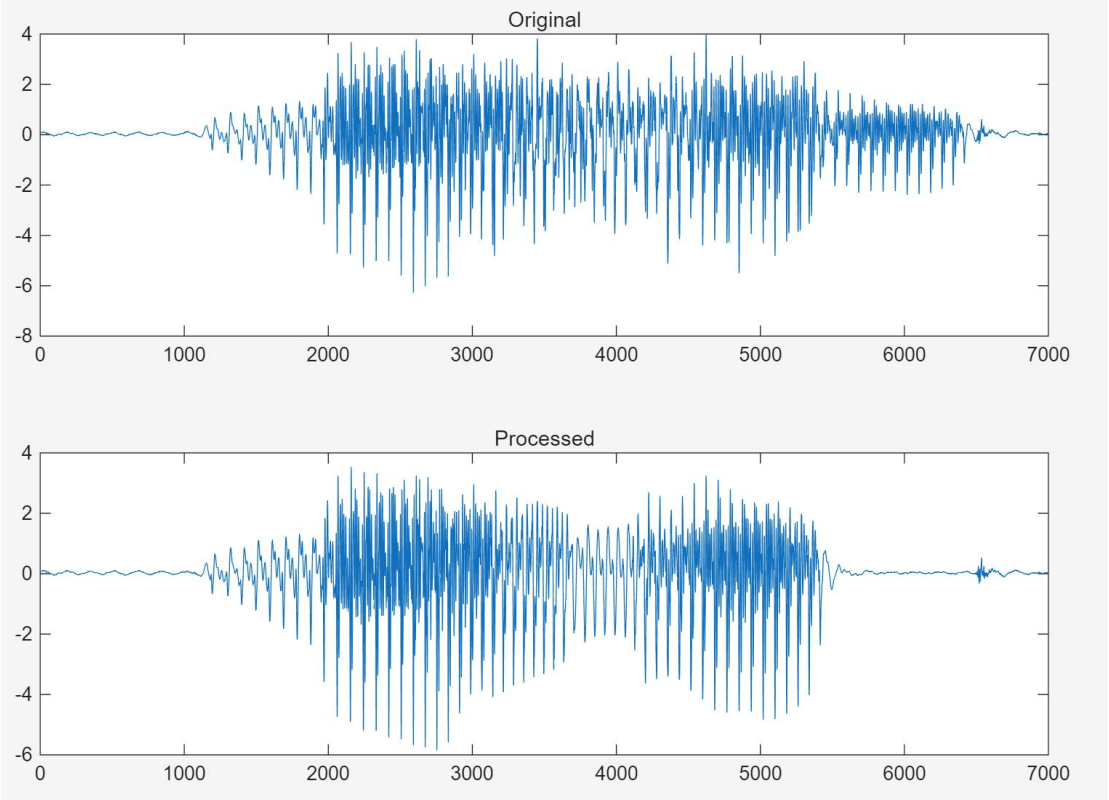


Figure 2 shows the inverse system response her. It is an Infinite Impulse Response (IIR). We can see a series of decaying spikes every 1000 samples. This feedback loop is designed to cancel out the repeating echoes step by step.

Output for (d)



We applied the filter to the real audio file lineup.mat. Figure 3 shows the waveforms before and after processing. The processed wave is much cleaner. When we play it with the sound() function, the echo is gone. This proves inverse filtering works perfectly in digital audio processing.

4 Experimental Conclusions

Through numerical simulation and analytical derivation, this experiment verified the core properties of discrete-time and continuous-time LTI systems.

(1)The linearity and time-invariance of a system are independent properties and can be visually verified by comparing output waveforms.

(2)The impulse response is the derivative of the step response. An ideal impulse can be accurately approximated by a finite-width pulse, provided the unit area is maintained.

(3)Difference equations can model signal distortion (like echoes) and can also be used to construct inverse IIR filters to recover the original signals, demonstrating the powerful engineering value of LTI system theory in digital signal processing.